1. Create a file TomStory.cpp
2. Create a class Brain with the following details:

```
class Brain {
    bool good = true; // by default, it is good
    string status = "";  // by default brain does nothing
public:
    void whatsup() {
        if (status != "") {
            cout << "Got an idea Sir ji, to trouble others, Ma'am ji!" << endl;
            return;
        }
        cout << "Getting bored doing C++" << endl;
    }
    bool isWicked() {
        return !good;
    }
    void think() {
        status = "Gibberish";
        good = false;
    }
    void rethink() {
        status = "";
        good = true;
    }
};
```

3. Add header inclusion line for iostream and namespace std at the top of the file.
   Compile to check whether there is any error.

4. Create a class **Spectacles,** *with one private attribute* **shopName** of string type (add #include<string> after the iostream inclusion line for compilation)**.**
   Initialize shopName to "E-city Gift Shop" in-place (wherever you are declaring the attribute, do = "E-city Gift Shop" there).
5. In the Spectacles class, create a public method **void show**(), which should do the following cout:

```
cout << "Got from "
     << shopName << "." << endl;
```

   Compile to check whether there is any error.

6. Create a class **Trap**, with private Boolean member **placed, initialized with false.**
7. Create a public method void **place()** in Trap class**, *which flip the placed flag*** and does the following cout:

```
cout << "The trap has been placed." << endl;
```

   Compile to check whether there is any error.

8. Create class **Cat** with the following private attributes:

```
// COMPOSITION:
// Cat has a brain. myBrain object (instance of class Brain) gets created when one Cat object gets created
string myName;
Brain myBrain;
bool sleeping = false;
bool wearingSpectacles = false;
// AGGREGATION:
// refers to independently existing Spectacles.
// When Tom, instance of this class, dies (goes out of scope), spectacles object still may exist
Spectacles* spectacles = nullptr;
```

9. Create the following methods inside the Cat class.
   - Private method **void whatsup():** it should call myBrain.whatsup();
   - Public method void setName(string name): it should assign name to myName attribute.
   - Public method as follows:

```
void wearSpectacles(Spectacles* sp) {
    if (sleeping) {
        cout << myName << " is sleeping and does not need spectacles."
             << endl;
    }
    spectacles = sp;
    wearingSpectacles = true;
}
```

   - Public method **removeSpectacles()**, which should just flip the corresponding flag.
   - Public method as follows:

```
bool canSee() {
    if (sleeping) {
        cout << myName << " is sleeping, so cannot see."
             << endl;
        return false;
    }
    if (spectacles != nullptr && wearingSpectacles) {
        cout << myName << " can see clearly." << endl;
        return true;
    }
    {
        cout << myName << " cannot see without spectacles."
             << endl;
        return false;
    }
}
```

   - Public method void sleep(), which sets sleeping flag, and makes the wearingSpectacles flag false.
   - Public method void wakeUp(), which makes the sleeping flag false, and if spectacles pointer is not null, then only makes the wearingSpectacles flag true.

Compile to check whether there is any error.

10. Create the next set of public methods in Cat class, as follows:

```
// ASSOCIATION:
// only uses the Trap. Do not own it.
// The Trap is not stored as a data member.
void torublePeopleUsingTrap(Trap* trap) {
    myBrain.think();
    trap->place(); // places the trap and forgets about it, so no need to store it as a class member --> association
}

void patpat() {
    myBrain.rethink(); // call delegation to compositional attributes
    if(myBrain.isWicked() == false)
        cout << myName << " is a good cat!" << endl;
}
void show() {
    cout << endl;
    if (spectacles != nullptr) {
        spectacles->show(); // call delegation to aggregated attributes
    }
    canSee();
    whatsup();
    cout << endl;
}
```

Compile to check whether there is any error.

11. Create the following main function below everything.

```
int main() {
    Cat tom;
    tom.setName( "Tom");
    tom.show();

    Spectacles spectacles;
    tom.wearSpectacles(&spectacles);
    tom.show();

    tom.sleep();
    tom.show();

    tom.wakeUp();
    tom.show();

    // Trap is also created independently.
    Trap stolenTrap;
    tom.torublePeopleUsingTrap(&stolenTrap);
    tom.show();

    tom.patpat();
    tom.show();

    return 0;
}
```

Compile and execute, and verify whether the program is behaving as expected.